---

1. SQL Functions (General Idea)

A SQL function takes input, processes it, and returns a value.

Examples:

SELECT UPPER('hello') -- HELLO

SELECT LEN('Database') -- 8

SELECT GETDATE() -- current date/time

The functions you asked about are **data conversion functions**.

They convert one datatype into another.

Example:

SELECT CAST('123' AS INT)

Converts text (VARCHAR) into a number (INT).

---

2. CAST

Purpose

Converts one datatype to another.

Syntax

CAST(expression AS datatype)

Example

SELECT CAST('100' AS INT)

Result:

100

Another example:

SELECT CAST(123.456 AS INT)

Result:

123

(decimal part removed)

When to use CAST

Use when:

- You need standard SQL

- Simple datatype conversion is enough
- Portability matters

---

3. CONVERT

Purpose

Also converts datatypes, but with extra formatting options.

Syntax

CONVERT(datatype, expression, style)

Example

SELECT CONVERT(INT, '100')

Result:

100

Date Formatting Example

SELECT CONVERT(VARCHAR, GETDATE(), 103)

Style 103 = British format.

Result:

19/05/2026

Common Date Styles

Style	Format
101	mm/dd/yyyy
102	yyyy.mm.dd
103	dd/mm/yyyy
120	yyyy-mm-dd hh:mi:ss

CAST vs CONVERT

Feature	CAST	CONVERT
ANSI standard	Yes	No (SQL Server specific)
Formatting support	No	Yes
Simpler syntax	Yes	Slightly more complex

---

4. PARSE

Purpose

Converts string values into date/number types using culture settings.

Syntax

PARSE(string_value AS datatype USING culture)

Example

SELECT PARSE('19/05/2026' AS DATE USING 'en-GB')

Result:

2026-05-19

Another Example

SELECT PARSE('1,234.56' AS MONEY USING 'en-US')

Important Notes

PARSE:

- Uses .NET framework internally
- Slower than CAST/CONVERT
- Mainly for formatted strings from users

---

5. TRY_CAST

Purpose

Like CAST, but safer.

If conversion fails:

- CAST → throws error

- TRY_CAST → returns NULL

Example

CAST

SELECT CAST('ABC' AS INT)

Result:

Error

TRY_CAST

SELECT TRY_CAST('ABC' AS INT)

Result:

NULL

Very Useful For Dirty Data

Example:

SELECT TRY_CAST(ColumnName AS INT)

FROM Table1

Bad values become NULL instead of crashing query execution.

---

6. TRY_CONVERT

Purpose

Safe version of CONVERT.

Example

SELECT TRY_CONVERT(INT, '500')

Result:

500

Bad conversion:

SELECT TRY_CONVERT(INT, 'Hello')

Result:

NULL

Supports Formatting Styles

SELECT TRY_CONVERT(DATE, '19/05/2026', 103)

---

7. TRY_PARSE

Purpose

Safe version of PARSE.

Returns NULL instead of error.

Example

SELECT TRY_PARSE('19/05/2026' AS DATE USING 'en-GB')

Valid result:

2026-05-19

Invalid:

SELECT TRY_PARSE('XYZ' AS INT)

Result:

NULL

Simple Analogy

Think of these like translators:

Function	Meaning
CAST	Basic translator
CONVERT	Translator with formatting
PARSE	Translator using regional language rules
TRY_CAST	Safe CAST
TRY_CONVERT	Safe CONVERT
TRY_PARSE	Safe PARSE

Performance Recommendation

In SQL Server:

Best Choice	Reason
CAST	Fastest/simple
CONVERT	Good for formatting
TRY_CAST	Safe + efficient
TRY_CONVERT	Safe + formatting
PARSE	Slowest
TRY_PARSE	Slowest safe version

Avoid PARSE in large datasets unless necessary.

Quick Comparison Table

Function	Error on Failure?	Formatting Support	Culture Support
CAST	Yes	No	No
CONVERT	Yes	Yes	Limited
PARSE	Yes	Yes	Yes
TRY_CAST	No (NULL)	No	No
TRY_CONVERT	No (NULL)	Yes	Limited
TRY_PARSE	No (NULL)	Yes	Yes

Real-World Example

Suppose a table stores salary as text:

EmployeeSalary

'5000'

'7000'

'ABC'

Using CAST:

SELECT CAST(EmployeeSalary AS INT)

FROM Employees

Fails because of 'ABC'.

Using TRY_CAST:

SELECT TRY_CAST(EmployeeSalary AS INT)

FROM Employees

Result:

5000

7000

NULL

Much safer for production systems.

Rule of Thumb

Use:

- CAST → normal conversion
- CONVERT → formatting dates/numbers
- TRY_CAST → uncertain data
- TRY_CONVERT → uncertain + formatting
- PARSE / TRY_PARSE → culture-specific string parsing only when needed